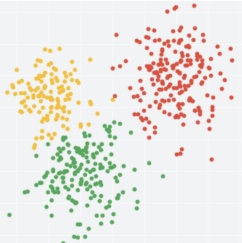
Logistic Regression

Procheta Sen



Probabilistic vs "ordinary" classifier

- **"Ordinary" classifier** is a function f that assigns to an input object $\mathbf{X}$ a predicted class c from a fixed set of classes $\{c_1, c_2, \ldots, c_k\}$, i.e.

$$c = f(\mathbf{X})$$

- **Probabilistic classifier** is a conditional distribution $P(C \mid \mathbf{X})$. For an input object $\mathbf{X}$ it gives probabilities $p_1, p_2, \ldots, p_k$, where

$$p_i = P(c_i \mid \mathbf{X})$$

and

$$p_1 + p_2 + \cdots + p_k = 1$$

Two types of models: discriminative vs generative

Discriminative

- Assume that the conditional distribution $P(C \mid X)$ (i.e. the probabilistic classifier) has specific form $P_\theta(C \mid X)$ depending on some parameters $\theta = (\theta_1, \ldots, \theta_k)$.
- Use training data set to **find / learn** parameters $\theta_1, \ldots, \theta_k$ such that the resulting distribution is "best possible" among all distributions of the assumed form.

Generative

- Assume that data come from specific distribution $P_\theta(X, C)$ depending on some parameters $\theta = (\theta_1, \ldots, \theta_k)$.
- Use training data set to **find / learn** parameters $\theta_1, \ldots, \theta_k$ such that the resulting distribution is "best possible" among all distributions of the assumed form.
- Use $P_\theta(X, C)$ to classify new objects.

Probabilistic classifiers

Generative

- Naive Bayes

$$P(H \mid E) = \frac{P(E, H)}{P(E)} = \frac{P(E \mid H)P(H)}{P(E)}$$

- ...

Discriminative

- **Logistic regression** (today!)
- Multilayer perceptrons (neural networks)
- ...

Setup

- We consider binary classification problems with classes $\{-1, +1\}$.
- We want to build a probabilistic classifier that outputs the probability of a particular training instance $\mathbf{X}$ being positive ($y = +1$) or negative ($y = -1$).

Logistic regression: main idea

- Define a separating hyperplane H parameterised by the feature weights $\mathbf{W} = (w_1, \ldots, w_d)$ and a bias parameter b, i.e.

$$H = \left\{ b + \sum_{i=1}^{d} w_i x_i = 0 \mid x_1, \ldots, x_d \right\}$$

- In perceptron, to classify an input object $\mathbf{X} = (x_1, \ldots, x_d)$, we used only the sign of

$$b + \mathbf{W}^T \mathbf{X} = b + \sum_{i=1}^{d} w_i x_i$$

which tells in which of the two half-spaces created by the hyperplane the point is located.

Logistic regression: main idea

- However, the actual value of $b + \mathbf{W}^T\mathbf{X}$ conveys extra useful information: it is proportional to the distance from point $\mathbf{X}$ to the hyperplane H.

- **Logistic regression** uses
  - sign of $b + \mathbf{W}^T\mathbf{X}$ to classify object $\mathbf{X}$, and
  - $|b + \mathbf{W}^T\mathbf{X}|$ to quantify our confidence in this classification: the larger the value, the further $\mathbf{X}$ from the separating hyperplane H.

Logistic sigmoid function

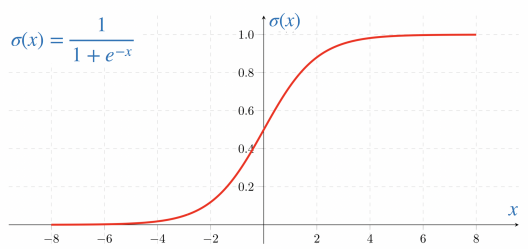


Figure: Caption

Logistic sigmoid function properties

- $\sigma(x) \in [0, 1]$ for any $x \in [-\infty, \infty]$
- $1 - \sigma(x) = \sigma(-x)$
- $$\frac{\partial \sigma}{\partial x} = \sigma(x) \cdot (1 - \sigma(x))$$

Logistic regression: discriminative classifier

Discriminative classifier

- Assume that the conditional distribution $P(C \mid X)$ (i.e. the probabilistic classifier) has specific form $P_\theta(C \mid X)$ depending on some parameters $\theta = (\theta_1, \ldots, \theta_k)$.

- Use training data set to **find / learn** parameters $\theta_1, \ldots, \theta_k$ such that the resulting distribution is "best possible" among all distributions of the assumed form.

Logistic regression: model assumption

- For an object $\mathbf{X} = (x_1, \ldots, x_d)$, the probability that $\mathbf{X}$ belongs to the positive class is modelled as

$$P(y = +1 \mid \mathbf{X}) = \sigma(a) = \frac{1}{1 + e^{-a}}$$

where $a = b + \mathbf{W}^T\mathbf{X}$. Hence the probability that $\mathbf{X}$ belongs to the negative class is

$$P(y = -1 \mid \mathbf{X}) = 1 - P(y = +1 \mid \mathbf{X}) = 1 - \sigma(a) = \sigma(-a) = \frac{1}{1 + e^{a}}$$

It is convenient to write

$$P(y = t \mid \mathbf{X}) = \sigma(t \cdot a) = \frac{1}{1 + e^{-ta}}, \quad \text{where } t \in \{-1, +1\}$$

Logistic regression: choosing/fitting parameters

- Let $\mathcal{D}^+ = \{(\mathbf{X}_1, y_1), (\mathbf{X}_2, y_2), \ldots, (\mathbf{X}_n, y_n)\}$ be the training data set.
- Using the **maximum likelihood estimation** method we would like to find parameters $b, w_1, w_2, \ldots, w_d$ that maximise the likelihood function

$$\ell(b, w_1, w_2, \ldots, w_d, \mathcal{D}) = \prod_{i=1}^{n} \sigma\Big(y_i \Big(b + \mathbf{W}^T \mathbf{X}_i\Big)\Big)$$

- This is equivalent to minimising $-\ell$ or minimising the negative log-likelihood function

$$-\ell = -\log \ell = -\sum_{i=1}^{n} \log \sigma\Big(y_i \Big(b + \mathbf{W}^T \mathbf{X}_i\Big)\Big)$$

Logistic regression: choosing/fitting parameters

$$-\ell = -\log \ell = -\sum_{i=1}^{n} \log \sigma \left(y_i \left(b + \mathbf{W}^T \mathbf{X}_i \right) \right)$$

$$\nabla_{b,w_1,\ldots,w_d}(-\ell) = -\nabla_{b,w_1,\ldots,w_d} \ell$$

Denote

$$a_i = b + \mathbf{W}^T \mathbf{X}_i$$

We need to compute

$$\frac{\partial \ell}{\partial b} \quad \text{and} \quad \frac{\partial \ell}{\partial w_k} \quad \text{for every } k = 1, \ldots, d.$$

Logistic regression: choosing/fitting parameters

$$\ell\ell = \log \ell = \sum_{i=1}^{n} \log \sigma\Big(y_i(b + \mathbf{W}^T \mathbf{X}_i)\Big) = \sum_{i=1}^{n} \log \sigma(y_i \cdot a_i)$$

$$\frac{\partial \ell\ell}{\partial b} = \sum_{i=1}^{n} y_i \cdot \sigma\Big(-y_i(b + \mathbf{W}^T \mathbf{X}_i)\Big) = \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i)$$

Logistic sigmoid function properties

1. $\dfrac{\partial \sigma}{\partial x} = \sigma(x) \cdot (1 - \sigma(x))$
2. $1 - \sigma(x) = \sigma(-x)$

Logistic regression: choosing/fitting parameters

$$\ell\ell = \log \ell = \sum_{i=1}^{n} \log \sigma\Big(y_i(b + \mathbf{W}^T \mathbf{X}_i)\Big) = \sum_{i=1}^{n} \log \sigma(y_i \cdot a_i)$$

$$\frac{\partial \ell\ell}{\partial w_k} = \sum_{i=1}^{n} y_i \cdot \sigma\Big(-y_i(b + \mathbf{W}^T \mathbf{X}_i)\Big) \cdot x_k^{(i)} = \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i) \cdot x_k^{(i)}$$

where

$$\mathbf{X}_i = (x_1^{(i)}, x_2^{(i)}, \ldots, x_d^{(i)})$$

Logistic regression: choosing/fitting parameters

$$\ell\ell = \log \ell = \sum_{i=1}^{n} \log \sigma\Big(y_i(b + \mathbf{W}^T \mathbf{X}_i)\Big) = \sum_{i=1}^{n} \log \sigma(y_i \cdot a_i)$$

Interpretation of

$$\textstyle\sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i)$$

- If $y_i = +1$, then
$$\sigma(-y_i \cdot a_i) = \sigma(-a_i) = 1 - \sigma(a_i) = P(y = -1 \mid \mathbf{X}_i)$$
- If $y_i = -1$, then
$$\sigma(-y_i \cdot a_i) = \sigma(a_i) = P(y = +1 \mid \mathbf{X}_i)$$
- Hence $\sigma(-y_i \cdot a_i)$ is the probability of misclassifying the training object $\mathbf{X}_i$.

$$\textstyle\sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i) = \sum_{\mathbf{X}_i \in \mathcal{D}_+} P(y = -1 \mid \mathbf{X}_i) - \sum_{\mathbf{X}_i \in \mathcal{D}_-} P(y = +1 \mid \mathbf{X}_i)$$

Logistic Regression: Update Rule

Gradient Descent method for finding local minimum of

$$f(\mathbf{Z}) = f(z_1, \ldots, z_d)$$

1. Pick an initial point $\mathbf{Z}_0$
2. Iterate according to

$$\mathbf{Z}_{i+1} = \mathbf{Z}_i - \gamma_i \left((\nabla_{\mathbf{Z}} f)(\mathbf{Z}_i) \right)$$

where $\gamma_1, \gamma_2, \ldots$ are step-sizes.

Logistic regression: update rule

$$\text{minimize } -\ell\ell = -\log \ell = -\sum_{i=1}^{n} \log \sigma\left(y_i(b + \mathbf{W}^T \mathbf{X}_i)\right)$$

$$\frac{\partial \ell\ell}{\partial b} = \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i) \qquad \frac{\partial \ell\ell}{\partial w_k} = \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i) \cdot x_k^{(i)}, \quad k = 1, \ldots, d$$

Hence we have the following update rule

$$b \leftarrow b + \mu \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i)$$

$$\mathbf{W} \leftarrow \mathbf{W} + \mu \sum_{i=1}^{n} y_i \cdot \sigma(-y_i \cdot a_i)\mathbf{X}_i$$

Online vs Batch

Batch

- Uses the **entire training dataset** in every iteration to update the weight vector
- Popular optimisation algorithm for the batch learning of logistic regression is the Limited Memory BFGS (L-BFGS) algorithm
- Batch version is slow compared to the online version, but often shows slightly improved accuracy in many cases

Online

- Uses only **one training object** in every iteration to update the weight vector
- The Stochastic Gradient Descent algorithm (SGD)
- SGD version can require multiple iterations over the dataset before it converges (if ever)
- SGD is frequently used with large-scale machine learning tasks (even when the objective function is non-convex)

Logistic regression online algorithm (Stochastic Gradient Descent)

LogisticRegression (Training data: $\{(\bar{X}_1, y_1), \ldots, (\bar{X}_n, y_n)\}$, Learning rate μ, MaxIter)

1. $w_i = 0$ for all $i = 1, \ldots, d$
2. $b = 0$
3. **for** iter $= 1 \ldots$ MaxIter **do**
4. **for** $i = 1 \ldots n$ **do**
5. $a_i = b + \bar{W}^T \bar{X}_i$
6. $w_s = w_s + \mu \cdot y_i \cdot \sigma(-y_i a_i) \cdot x_s^{(i)}$ for all $s = 1, \ldots, d$
7. $b = b + \mu \cdot y_i \cdot \sigma(-y_i a_i)$
8. **return** $b, w_1, w_2, \ldots, w_d$

Update rule

$$\bar{W} \leftarrow \bar{W} + \mu \cdot y_i \cdot \sigma(-y_i a_i) \bar{X}_i$$

$$b \leftarrow b + \mu \cdot y_i \cdot \sigma(-y_i a_i)$$

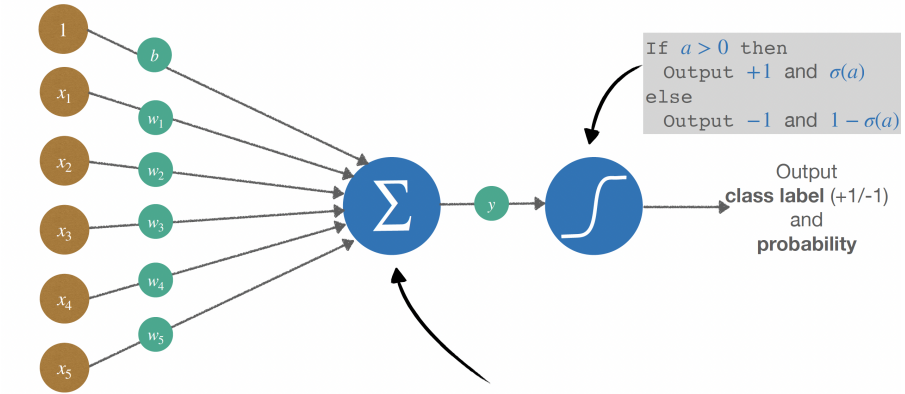
Logistic Regression Prediction

`LogisticRegressionTest(`$b, w_1, w_2, \ldots, w_d, \bar{X}$`)`

1. $a = b + \bar{W}^T \bar{X}$
2. **if** $a > 0$ **then**
   - predicted label $= +1$ `# positive class`
   - probability that $\bar{X}$ belongs to the positive class $= \sigma(a)$ `# confidence`
3. **else**
   - predicted label $= -1$ `# negative class`
   - probability that $\bar{X}$ belongs to the negative class $= 1 - \sigma(a)$ `# confidence`

Neuron Interpretation of Logistic Regression

Logistic regression: neuron interpretation



Logistic Regression Prediction

`LogisticRegressionTest(`$b, w_1, w_2, \ldots, w_d, \bar{X}$`)`

1. $a = b + \bar{W}^T \bar{X}$
2. **if** $a > 0$ **then**
   - predicted label $= +1$ `# positive class`
   - probability that $\bar{X}$ belongs to the positive class $= \sigma(a)$ `# confidence`
3. **else**
   - predicted label $= -1$ `# negative class`
   - probability that $\bar{X}$ belongs to the negative class $= 1 - \sigma(a)$ `# confidence`

L2 regularisation in logistic regression

L2 regularised logistic regression update rule for training object $(\bar{X}, y)$ with

$$a = b + \bar{W}^T \bar{X}$$

No regularisation:

$$\bar{W} \leftarrow \bar{W} + \mu \cdot y \cdot \sigma(-y \cdot a) \cdot \bar{X}$$

With regularisation:

$$\bar{W} \leftarrow \bar{W} - \mu \cdot \left(-y \cdot \sigma(-y \cdot a) \cdot \bar{X} + 2\lambda \bar{W}\right)$$

$$= \bar{W} + \mu \cdot y \cdot \sigma(-y \cdot a) \cdot \bar{X} - 2\mu\lambda \bar{W}$$

$$= (1 - 2\mu\lambda)\bar{W} + \mu \cdot y \cdot \sigma(-y \cdot a) \cdot \bar{X}$$